

데이터 정제하고 병합하기



데이터 정제하기

- 원본(Raw) 데이터는 불완전하거나 부정확한 경우가 많음
- 데이터가 누락되어 있거나, 잘못된 값이 포함되어 있는 경우 등 다양한 문제가 존재
- 데이터를 제거하거나 대체하는 등 다양한 정제과정이 필요



데이터 정렬: sort_values()

DataFrame의 특정 컬럼의 값을 기준으로 전체 데이터를 정렬할 때 사용
ascending이 True이면 오름차순, False면 내림차순

```
df.sort_values("컬럼 이름", ascending = True 또는 False)
```

```
df.sort_values("총계")
```

	날짜	공휴일	날씨	유료합계	어른	청소년	어린이	외국인	단체	무료합계	총계	연	월	일	요일	매출액
352	2017-03-28	X	구름 많음	0	0	0.0	0	0	0	0	2	2017	3	28	화	0
353	2017-03-29	X	구름 조금	0	0	0.0	0	0	0	0	7	2017	3	29	수	0
654	2018-01-24	X	맑음	18	16	1.0	1	0	0	11	128	2018	1	24	수	180000
655	2018-01-25	X	맑음	23	23	0.0	0	1	0	13	162	2018	1	25	목	230000
656	2018-01-26	X	맑음	50	31	13.0	6	0	0	17	222	2018	1	26	금	500000
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
363	2017-04-08	O	구름 조금	28514	24756	638.0	3120	220	137	1237	40887	2017	4	8	토	285140000
156	2016-06-05	O	구름 조금	33504	28943	1079.0	3482	729	190	8239	41743	2016	6	5	일	335040000
390	2017-05-05	O	구름 조금	27046	25824	1222.0	0	616	170	1921	46264	2017	5	5	금	270460000
755	2018-05-05	O	맑음	29521	28592	899.0	30	208	65	2278	52310	2018	5	5	토	295210000
125	2016-05-05	O	NaN	40219	38511	1708.0	0	476	35	18469	58688	2016	5	5	목	402190000

```
df.sort_values("총계", ascending=False)
```

	날짜	공휴일	날씨	유료합계	어른	청소년	어린이	외국인	단체	무료합계	총계	연	월	일	요일	매출액
125	2016-05-05	O	NaN	40219	38511	1708.0	0	476	35	18469	58688	2016	5	5	목	402190000
755	2018-05-05	O	맑음	29521	28592	899.0	30	208	65	22789	52310	2018	5	5	토	295210000
390	2017-05-05	O	구름 조금	27046	25824	1222.0	0	616	170	19213	46264	2017	5	5	금	270460000
156	2016-06-05	O	구름 조금	33504	28943	1079.0	3482	729	190	8239	41743	2016	6	5	일	335040000
363	2017-04-08	O	구름 조금	28514	24756	638.0	3120	220	137	12373	40887	2017	4	8	토	285140000
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
656	2018-01-26	X	맑음	50	31	13.0	6	0	0	172	222	2018	1	26	금	500000
655	2018-01-25	X	맑음	23	23	0.0	0	1	0	139	162	2018	1	25	목	230000
654	2018-01-24	X	맑음	18	16	1.0	1	0	0	110	128	2018	1	24	수	180000
353	2017-03-29	X	구름 조금	0	0	0.0	0	0	0	7	7	2017	3	29	수	0
352	2017-03-28	X	구름 많음	0	0	0.0	0	0	0	2	2	2017	3	28	화	0



인덱스 재지정: reset_index()

- 데이터프레임의 인덱스를 처음부터 다시 지정해줄 때 사용
- 데이터프레임을 정렬하는 등의 작업을 하다 보면 index가 뒤섞이게 되는데,
이 때 reset_index()를 활용하여 인덱스 초기화

```
df.reset_index()
```

	index	날짜	공휴일	날씨	유료합계	어른	청소년	어린이	외국인	단체	무료합계	총계	연	월	일	요일	매출액
0	125	2016-05-05	O	NaN	40219	38511	1708.0	0	476	35	18469	58688	2016	5	5	목	402190000
1	755	2018-05-05	O	맑음	29521	28592	899.0	30	208	65	22789	52310	2018	5	5	토	295210000
2	390	2017-05-05	O	구름 조금	27046	25824	1222.0	0	616	170	19218	46264	2017	5	5	금	270460000
3	156	2016-06-05	O	구름 조금	33504	28943	1079.0	3482	729	190	8239	41743	2016	6	5	일	335040000
4	363	2017-04-08	O	구름 조금	28514	24756	638.0	3120	220	137	12373	40887	2017	4	8	토	285140000
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1081	656	2018-01-26	X	맑음	50	31	13.0	6	0	0	172	222	2018	1	26	금	500000
1082	655	2018-01-25	X	맑음	23	23	0.0	0	1	0	139	162	2018	1	25	목	230000
1083	654	2018-01-24	X	맑음	18	16	1.0	1	0	0	110	128	2018	1	24	수	180000
1084	353	2017-03-29	X	구름 조금	0	0	0.0	0	0	0	7	7	2017	3	29	수	0
1085	352	2017-03-28	X	구름 많음	0	0	0.0	0	0	0	2	2	2017	3	28	화	0

index가 초기화 되면서
기존의 index를 저장한 "index"컬럼이 생김

```
df.reset_index(drop=True)
```

	날짜	공휴일	날씨	유료합계	어른	청소년	어린이	외국인	단체	무료합계	총계	연	월	일	요일	매출액
0	2016-05-05	O	NaN	40219	38511	1708.0	0	476	35	18469	58688	2016	5	5	목	402190000
1	2018-05-05	O	맑음	29521	28592	899.0	30	208	65	22789	52310	2018	5	5	토	295210000
2	2017-05-05	O	구름 조금	27046	25824	1222.0	0	616	170	19218	46264	2017	5	5	금	270460000
3	2016-06-05	O	구름 조금	33504	28943	1079.0	3482	729	190	8239	41743	2016	6	5	일	335040000
4	2017-04-08	O	구름 조금	28514	24756	638.0	3120	220	137	12373	40887	2017	4	8	토	285140000
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1081	2018-01-26	X	맑음	50	31	13.0	6	0	0	172	222	2018	1	26	금	500000
1082	2018-01-25	X	맑음	23	23	0.0	0	1	0	139	162	2018	1	25	목	230000
1083	2018-01-24	X	맑음	18	16	1.0	1	0	0	110	128	2018	1	24	수	180000
1084	2017-03-29	X	구름 조금	0	0	0.0	0	0	0	7	7	2017	3	29	수	0
1085	2017-03-28	X	구름 많음	0	0	0.0	0	0	0	2	2	2017	3	28	화	0



데이터 삭제: drop()

데이터프레임에서 특정 행이나 열을 삭제할 때 사용

```
df.drop("행/열 이름", axis=0 또는 1)
```

axis=0이면 행 삭제, axis=1이면 열 삭제

```
df=df.drop(["유료합계", "무료합계"], axis=1)
```

df에서 "유료합계", "무료합계" 컬럼(axis=1이므로)을 삭제하고 df에 저장(덮어쓰우기)

	날짜	공휴일	날씨	유료합계	어른	청소년	어린이	외국인	단체	무료합계	총계	연	월	일	요일	매출액
0	2016-01-01	O	구름 조금	359	2799	141.0	419	47	0	1023	4382	2016	1	1	금	33590000
1	2016-01-02	O	구름 많음	573	4370	203.0	600	100	111	2092	7265	2016	1	2	토	51730000
2	2016-01-03	O	구름 많음	308	2571	128.0	309	91	0	1549	4557	2016	1	3	일	30080000
3	2016-01-04	X	구름 많음	90	602	NaN	235	51	223	890	1690	2016	1	4	월	8900000
4	2016-01-05	X	구름 많음	16	319	35.0	62	43	47	840	1256	2016	1	5	화	4160000



	날짜	공휴일	날씨	어른	청소년	어린이	외국인	단체	총계	연	월	일	요일	매출액
0	2016-01-01	O	구름 조금	2799	141.0	419	47	0	4382	2016	1	1	금	33590000
1	2016-01-02	O	구름 많음	4370	203.0	600	100	111	7265	2016	1	2	토	51730000
2	2016-01-03	O	구름 많음	2571	128.0	309	91	0	4557	2016	1	3	일	30080000
3	2016-01-04	X	구름 많음	602	NaN	235	51	223	1690	2016	1	4	월	8900000
4	2016-01-05	X	구름 많음	319	35.0	62	43	47	1256	2016	1	5	화	4160000



열 이름 바꾸기: rename()

데이터프레임의 특정 열 이름을 바꾸는데 사용

```
df.rename(columns = {"바꿀 컬럼 이름" : "새 컬럼 이름"})
```

```
df=df.rename(columns = {"총계" : "총입장객수"})
```

df에서 "총계" 컬럼의 이름을 "총입장객수"로 변환한 뒤 df에 저장(덮어쓰우기)

	날짜	공휴일	날씨	어른	청소년	어린이	외국인	단체	총계	연	월	일	요일
0	2016-01-01	O	구름 조금	2799	141.0	419	47	0	4382	2016	1	1	금
1	2016-01-02	O	구름 많음	4370	203.0	600	100	111	7265	2016	1	2	토
2	2016-01-03	O	구름 많음	2571	128.0	309	91	0	4557	2016	1	3	일
3	2016-01-04	X	구름 많음	602	NaN	235	51	223	1690	2016	1	4	월
4	2016-01-05	X	구름 많음	319	35.0	62	43	47	1256	2016	1	5	화



	날짜	공휴일	날씨	어른	청소년	어린이	외국인	단체	총입장객수	연	월	일	요일	매출액
0	2016-01-01	O	구름 조금	2799	141.0	419	47	0	4382	2016	1	1	금	33590000
1	2016-01-02	O	구름 많음	4370	203.0	600	100	111	7265	2016	1	2	토	51730000
2	2016-01-03	O	구름 많음	2571	128.0	309	91	0	4557	2016	1	3	일	30080000
3	2016-01-04	X	구름 많음	602	NaN	235	51	223	1690	2016	1	4	월	8900000
4	2016-01-05	X	구름 많음	319	35.0	62	43	47	1256	2016	1	5	화	4160000



결측치 탐색: isnull()

- 데이터프레임의 각 원소가 결측치인지 여부를 True/False로 반환할 때 사용
- sum()과 함께 사용하여 각 컬럼별로 결측치가 몇 개 존재하는지 확인하는 용도로 사용
- isna()와 동일한 함수

```
df.isnull()
```

	날짜	공휴일	날씨	유료함께	어른	청소년	어린이	외국인	단체	무료함께	총계	연	월	일	요일	매출액
0	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	True	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1081	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
1082	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
1083	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
1084	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False
1085	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False	False

데이터가 커서 결측치가 있는지 일일이 찾기 어려움

```
df.isnull().sum()
```

```
날짜      0
공휴일    0
날씨      0
어른      0
청소년    5
어린이    0
외국인    0
단체      0
총입장객수 0
연        0
월        0
일        0
요일      0
매출액    0
dtype: int64
```



결측치 채우기: fillna()

데이터프레임 내의 결측치를 다른 값으로 대체하기 위해 사용

```
df.fillna(대체 값)
```

2. df의 "청소년"컬럼의 평균을 구한 뒤 정수로 변환

```
df["청소년"] = df["청소년"].fillna(int(df["청소년"].mean()))
```

1. fillna를 사용하여 "청소년"컬럼에 존재하는 결측치를 채워 넣음

3. 결측치가 다 채워진 컬럼을 df의 "청소년"컬럼에 저장(덮어쓰우기)

결측치 데이터 삭제하기: dropna()

데이터프레임 내의 결측치가 포함된 행을 삭제하는데 사용

```
df.dropna(subset="컬럼이름", ignore_index=True 또는 False)
```

- subset: 특정 컬럼에 결측치가 존재하는지를 검사 (생략 시 전체 컬럼)
- ignore_index: True면 결측치가 존재하는 행이 삭제된 데이터프레임의 인덱스 초기화, False면 그대로(기본값은 False)

```
df=df.dropna(subset=["청소년"], ignore_index=True)
```

df에서 "청소년" 컬럼에 결측치가 있는 행을 삭제한 뒤 인덱스를 초기화하여
df에 저장(덮어쓰우기)

중복 데이터 처리: drop_duplicates()

동일한 행(중복)을 찾아 제거할 때 사용

```
df.duplicated()
```

중복행 여부를 True / False로 반환

```
df.duplicated().sum()
```

중복 행이 몇 개인지 확인

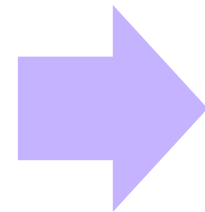
```
df = df.drop_duplicates()
```

중복 행 제거

데이터 병합하기

- 데이터가 여러 개로 분산되어 있거나, 추가 데이터를 합치고 싶은 경우
- 데이터 병합을 통해 데이터의 일관성을 유지하고 분석 결과의 신뢰도를 높일 수 있음

index	국어	수학	영어
민수	95	88	96
도윤	72	72	68
서아	83	93	85
연우	89	80	94
하준	100	95	93



index	국어	수학	영어
민수	95	88	96
도윤	72	72	68
서아	83	93	85
연우	89	80	94
하준	100	95	93
선우	95	92	97

index	국어	수학	영어	과학
민수	95	88	96	94
도윤	72	72	68	85
서아	83	93	85	90
연우	89	80	94	90
하준	100	95	93	88



데이터 합치기: concat()

공통의 컬럼을 가지는 데이터프레임을 합치는데 사용

```
pd.concat([데이터프레임1, 데이터프레임2], axis = 0 또는 1,  
          join = 'inner' 또는 'outer', ignore_index = True 또는 False)
```

- axis가 0이면 위아래로, 1이면 좌우로 합침 (기본값은 0)
- join이 'outer'이면 컬럼을 합집합으로, 'inner'이면 교집합으로 처리 (기본값은 outer)
- ignore_index가 False면 원본 데이터프레임의 인덱스를 유지
 - True면 기존의 인덱스를 무시하고 새로운 인덱스를 부여 (기본값은 False)



데이터 합치기: concat()

	날짜	공휴일	날씨	어른	청소년	어린이	외국인	단체	총입장객수	연	월	일	요일	매출액
0	2016-01-01	O	구름 조금	2799	141.0	419	47	0	4382	2016	1	1	금	33590000
1	2016-01-02	O	구름 많음	4370	203.0	600	100	111	7265	2016	1	2	토	51730000
2	2016-01-03	O	구름 많음	2571	128.0	309	91	0	4557	2016	1	3	일	30080000
3	2016-01-04	X	구름 많음	602	463.0	235	51	223	1690	2016	1	4	월	8900000
4	2016-01-05	X	구름 많음	319	35.0	62	43	47	1256	2016	1	5	화	4160000
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1081	2019-03-27	X	구름 많음	464	10.0	30	21	0	1117	2019	3	27	수	5040000
1082	2019-03-28	X	구름 많음	687	46.0	28	35	108	1665	2019	3	28	목	7610000
1083	2019-03-29	X	구름 조금	1447	120.0	77	14	188	2870	2019	3	29	금	16440000
1084	2019-03-30	O	흐림	1326	44.0	169	29	115	2452	2019	3	30	토	15390000
1085	2019-03-31	O	구름 조금	2563	111.0	387	53	0	4418	2019	3	31	일	30610000

df

2016년 1월 1일부터 2019년 3월 31일까지의 데이터
(폐쇄기간 제외)

	날짜	요일	공휴일	날씨	유료합계	어른	청소년	어린이	외국인	단체	무료합계	총계
0	2019-04-01	월	X	구름 조금	1,064	953	12	72	42	0	812	1,818
1	2019-04-02	화	X	구름 조금	1,020	938	4	34	36	55	1,061	1,958
2	2019-04-03	수	X	맑음	2,749	2,313	57	337	31	1651	1,208	3,813
3	2019-04-04	목	X	맑음	1,630	1472	25	74	50	123	1,430	2755
4	2019-04-05	금	X	맑음	2,991	1507	1306	97	21	1368	1,109	3872

df2

2019년 4월 1일부터 2019년 4월 30일까지의 데이터



데이터 합치기: concat()

```
df_concat = pd.concat([df, df2], axis=0, join="outer", ignore_index=True)
```

df와 df2를 병합한 데이터프레임을 df_concat에 저장

axis=0이므로 세로 방향으로, join이 'outer'이므로

두 데이터프레임의 컬럼을 모두 포함,

ignore_index는 True이므로 인덱스를 다시 부여

	날짜	공휴일	날씨	어른	청소년	어린이	외국인	단체	요일
0	2016-01-01 00:00:00	O	구름 조금	2799	141.0	419	47	0	금
1	2016-01-02 00:00:00	O	구름 많음	4370	203.0	600	100	111	토
2	2016-01-03 00:00:00	O	구름 많음	2571	128.0	309	91	0	일
3	2016-01-04 00:00:00	X	구름 많음	602	463.0	235	51	223	월
4	2016-01-05 00:00:00	X	구름 많음	319	35.0	62	43	47	화
...	...	...	...	...	...	...	...	...	...
1111	2019-04-26	X	구름 많음	1719	270.0	291	17	1911	금
1112	2019-04-27	O	구름 조금	15218	207.0	2173	125	447	토
1113	2019-04-28	O	구름 많음	10947	178.0	1780	88	455	일
1114	2019-04-29	X	흐림	1,169	47.0	130	27	370	월
1115	2019-04-30	X	NaN	1,910	378.0	381	63	2070	화

df_concat

2016년 1월 1일부터 2019년 4월 30일까지의 데이터
(폐쇄기간 제외)



데이터 합치기: merge()

특정 컬럼을 기준으로 합치는데 사용

```
pd.merge(데이터프레임1, 데이터프레임2, on="컬럼이름",  
         how='inner' 또는 'outer' 또는 'left' 또는 'right')
```

- on은 데이터프레임을 합칠 때 기준(key)이 되는 컬럼의 이름
- how는 데이터프레임의 데이터(행)를 합치는 방법으로 4가지 방법이 존재
 - inner: 두 데이터프레임에 모두 존재하는 데이터만 (교집합)
 - outer: 두 데이터프레임의 모든 데이터 사용 (합집합)
 - left: 데이터프레임1에 있는 모든 데이터
 - right: 데이터프레임2에 있는 모든 데이터



예시로 보는 merge()

index	월	평균 입장객수
0	3	2904
1	4	4035
2	5	5871
3	6	3661

index	월	미세먼지
0	5	85
1	6	73
2	7	54
3	8	60



예시로 보는 merge()

index	월	평균 입장객수
0	3	2904
1	4	4035
2	5	5871
3	6	3661

index	월	미세먼지
0	5	85
1	6	73
2	7	54
3	8	60

```
outer_df = pd.merge(df1, df2, on="월", how="outer")
```

```
inner_df = pd.merge(df1, df2, on="월", how="inner")
```

index	월	평균 입장객수	미세먼지
0	5	5871	85
1	6	3661	73

index	월	평균 입장객수	미세먼지
0	3	2904	NaN
1	4	4035	NaN
2	5	5871	85
3	6	3661	73
4	7	NaN	54
5	8	NaN	60



예시로 보는 merge()

index	월	평균 입장객수
0	3	2904
1	4	4035
2	5	5871
3	6	3661

index	월	미세먼지
0	5	85
1	6	73
2	7	54
3	8	60

```
left_df = pd.merge(df1, df2, on="월", how="left")
```

index	월	평균 입장객수	미세먼지
0	3	2904	NaN
1	4	4035	NaN
2	5	5871	85
3	6	3661	73

```
right_df = pd.merge(df1, df2, on="월", how="right")
```

index	월	평균 입장객수	미세먼지
0	5	5871	85
1	6	3661	73
2	7	NaN	54
3	8	NaN	60



데이터 합치기: merge()

	날짜	공휴일	날씨	어른	청소년	어린이	외국인	단체	총입장객수	연	월	일	요일
0	2016-01-01	O	구름 조금	2799	141.0	419	47	0	4382	2016	1	1	금
1	2016-01-02	O	구름 많음	4370	203.0	600	100	111	7265	2016	1	2	토
2	2016-01-03	O	구름 많음	2571	128.0	309	91	0	4557	2016	1	3	일
3	2016-01-04	X	구름 많음	602	463.0	235	51	223	1690	2016	1	4	월
4	2016-01-05	X	구름 많음	319	35.0	62	43	47	1256	2016	1	5	화
...	...	...	...	...	...	...	...	...	...	...	...	...	...
1081	2019-03-27	X	구름 많음	464	10.0	30	21	0	1117	2019	3	27	수
1082	2019-03-28	X	구름 많음	687	46.0	28	35	108	1665	2019	3	28	목
1083	2019-03-29	X	구름 조금	1447	120.0	77	14	188	2870	2019	3	29	금
1084	2019-03-30	O	흐림	1326	44.0	169	29	115	2452	2019	3	30	토
1085	2019-03-31	O	구름 조금	2563	111.0	387	53	0	4418	2019	3	31	일

df

2016년 1월 1일부터 2019년 3월 31일까지의
서울대공원 입장객 데이터 (폐쇄기간 제외)

	날짜	미세먼지	초미세먼지	오존	이산화질소	일산화탄소	아황산가스
0	2016-01-01	68	45	0.006	0.047	1.0	0.006
1	2016-01-02	51	31	0.009	0.044	0.7	0.006
2	2016-01-03	96	58	0.013	0.049	0.9	0.007
3	2016-01-04	74	47	0.025	0.025	0.6	0.006
4	2016-01-05	32	16	0.016	0.025	0.4	0.006
...	...	...	...	...	...	...	...
1181	2019-03-27	94	60	0.048	0.027	0.6	0.005
1182	2019-03-28	81	49	0.034	0.027	0.5	0.004
1183	2019-03-29	39	19	0.037	0.023	0.4	0.004
1184	2019-03-30	35	24	0.037	0.018	0.4	0.003
1185	2019-03-31	42	26	0.037	0.013	0.4	0.003

mm

2016년 1월 1일부터 2019년 3월 31일까지의
미세먼지 데이터



데이터 합치기: merge()

```
df_merge = pd.merge(df, mm, on="날짜", how="left")
```

df와 mm을 병합한 DataFrame을 df_merge에 저장
(on이 "날짜" 이므로 "날짜" 컬럼을 기준으로, how가 'left'이므로 왼쪽에 위치한 df의 날짜 컬럼을 기준으로 병합)

	날짜	공휴일	날씨	어른	청소년	어린이	외국인	단체	총입장객수	연	월	일	요일
347	2016-12-13	X	NaN	301	40.0	8	30	0	1312	2016	12	13	화
348	2016-12-14	X	NaN	208	15.0	28	13	137	930	2016	12	14	수
349	2016-12-15	X	NaN	101	18.0	4	13	0	698	2016	12	15	목
350	2016-12-16	X	구름 조금	129	14.0	12	9	0	565	2016	12	16	금
351	2016-12-17	O	구름 많음	1087	74.0	140	24	80	2403	2016	12	17	토
352	2017-03-28	X	구름 많음	0	0.0	0	0	0	2	2017	3	28	화
353	2017-03-29	X	구름 조금	0	0.0	0	0	0	7	2017	3	29	수
354	2017-03-30	X	구름 조금	1187	13.0	102	96	115	2369	2017	3	30	목
355	2017-03-31	X	눈/비	386	328.0	13	41	270	1299	2017	3	31	금
356	2017-04-01	O	눈/비	11155	204.0	1455	100	30	17070	2017	4	1	토

중요한 것은 미세먼지가 아닌 입장객 수

서울대공원이 폐쇄된 기간의 미세먼지 데이터는 의미가 없기에 how를 left로 설정하여 입장객 데이터가 존재하는 날짜만을 남김



데이터 합치기: merge()

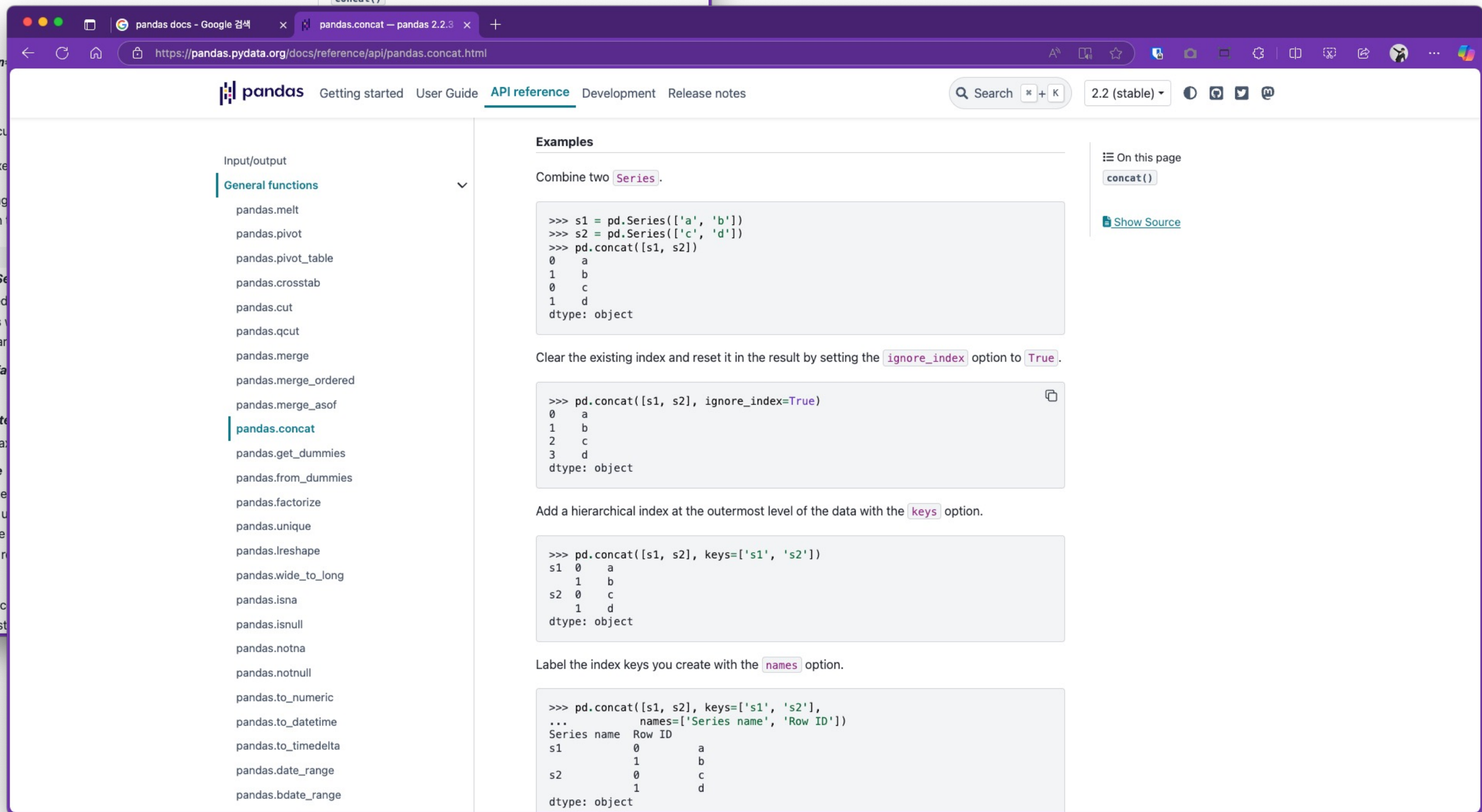
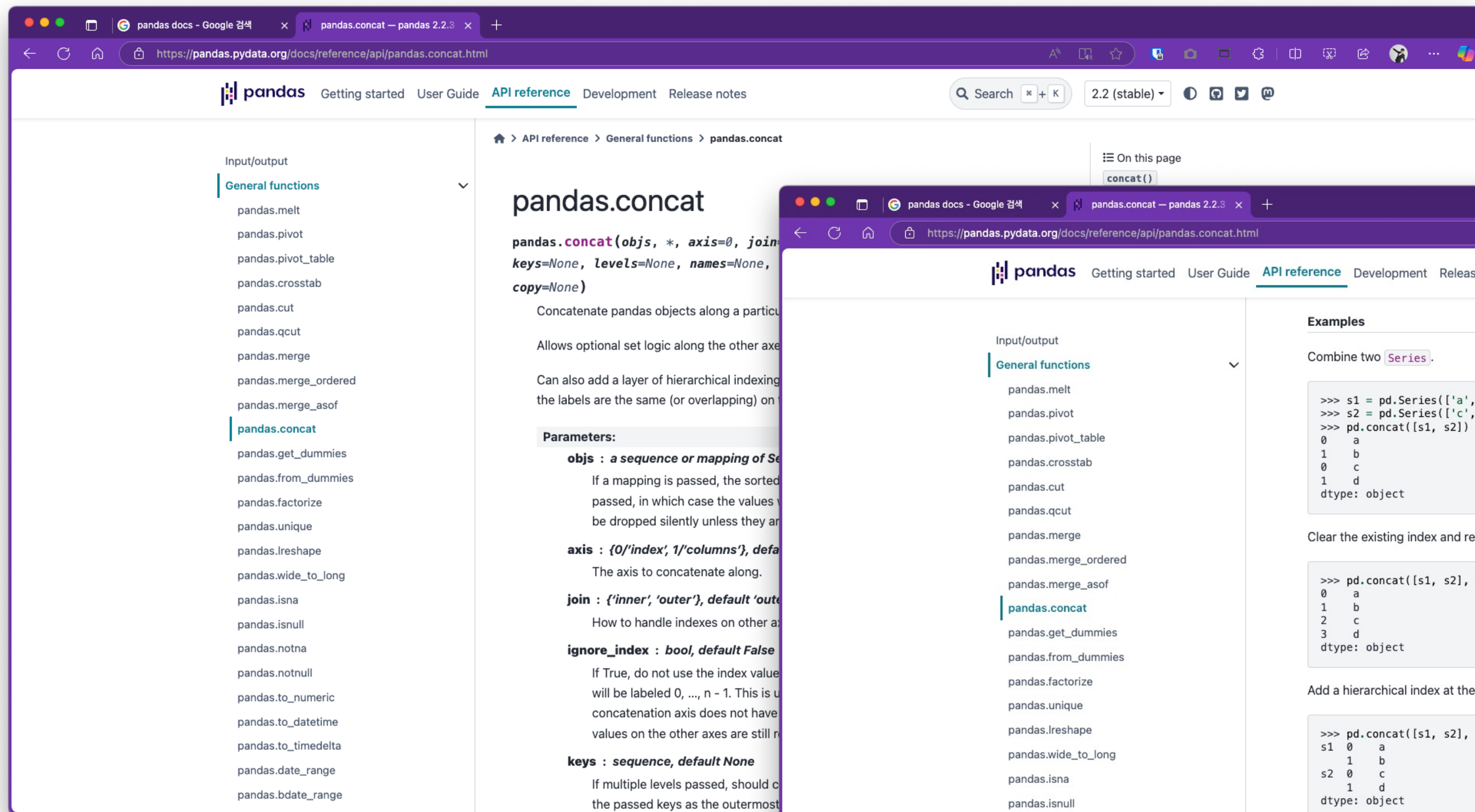
	날짜	공휴일	날씨	어른	청소년	어린이	외국인	단체	총입장객수	연	월	일	요일	미세먼지	초미세먼지	오존	이산화질소	일산화탄소	아황산가스
0	2016-01-01	O	구름 조금	2799	141.0	419	47	0	4382	2016	1	1	금	68	45	0.006	0.047	1.0	0.006
1	2016-01-02	O	구름 많음	4370	203.0	600	100	111	7265	2016	1	2	토	51	31	0.009	0.044	0.7	0.006
2	2016-01-03	O	구름 많음	2571	128.0	309	91	0	4557	2016	1	3	일	96	58	0.013	0.049	0.9	0.007
3	2016-01-04	X	구름 많음	602	463.0	235	51	223	1690	2016	1	4	월	74	47	0.025	0.025	0.6	0.006
4	2016-01-05	X	구름 많음	319	35.0	62	43	47	1256	2016	1	5	화	32	16	0.016	0.025	0.4	0.006
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1081	2019-03-27	X	구름 많음	464	10.0	30	21	0	1117	2019	3	27	수	94	60	0.048	0.027	0.6	0.005
1082	2019-03-28	X	구름 많음	687	46.0	28	35	108	1665	2019	3	28	목	81	49	0.034	0.027	0.5	0.004
1083	2019-03-29	X	구름 조금	1447	120.0	77	14	188	2870	2019	3	29	금	39	19	0.037	0.023	0.4	0.004
1084	2019-03-30	O	흐림	1326	44.0	169	29	115	2452	2019	3	30	토	35	24	0.037	0.018	0.4	0.003
1085	2019-03-31	O	구름 조금	2563	111.0	387	53	0	4418	2019	3	31	일	42	26	0.037	0.013	0.4	0.003

df_merge

2016년 1월 1일부터 2019년 3월 31일까지의 서울대공원 입장객 수와 미세먼지 데이터
(폐쇄기간 제외)



참고: 판다스 docs 확인



<https://pandas.pydata.org/docs/reference/api/pandas.concat.html>

(참고) 데이터 분석에서의 자료형 활용 : 튜플

구분	설명	예시
고정된 데이터 저장	수정되지 않아야 하는 데이터를 저장할 때 사용	좌표, RGB 색상 값, 날짜 등
함수 반환값 처리	함수에서 여러 개의 값을 반환할 때 사용	데이터의 최솟값, 최댓값을 함께 반환
딕셔너리의 키로 사용	여러 값을 결합하여 하나의 고정된 키로 사용	
의도치 않은 데이터 변경 방지	안정성이 요구되는 데이터 분석 과정에서 사용	

(참고) 데이터 분석에서의 자료형 활용 : 튜플

```
# 튜플 사용 예시
dimensions = (1920, 1080) # 고정된 화면 해상도
rgb_color = (255, 0, 0)   # 고정된 색상 값

# 함수에서 여러 값 반환
def get_min_max(data):
    return (min(data), max(data))

result = get_min_max([10, 2, 8, 6])
# result는 (2, 10) 형태의 튜플
```

(참고) 데이터 분석에서의 자료형 활용 : 딕셔너리

구분	설명	예시
빠른 데이터 조회	많은 양의 데이터를 다룰 때	이름을 키로 하고 점수를 값으로 저장한 경우
데이터 매핑	서로 관련된 데이터를 매핑할 때	고객 ID를 키로하고, 고객 정보를 값으로 저장
카운팅 or 그룹화	값으로 리스트나 딕셔너리 저장 가능하여 복잡한 데이터 모델을 표현해야 할 때	중첩된 데이터, JSON 형식 데이터
데이터 프레임과의 상호작용	Pandas의 DataFrame, Series와 호환	데이터를 딕셔너리 형으로 변환하고, 이를 Pandas DataFrame으로 변환해 분석



(참고) 데이터 분석에서의 자료형 활용 : 딕셔너리

```
# 딕셔너리 사용 예시
student_scores = {
    "Alice": 85,
    "Bob": 90,
    "Charlie": 78
}

# 값 조회
print(student_scores["Alice"]) # 출력: 85

# 데이터 추가 또는 수정
student_scores["David"] = 92

# 값 제거
del student_scores["Charlie"]

# 복잡한 데이터 구조
person = {
    "name": "John",
    "age": 30,
    "skills": ["Python", "Data Analysis", "Machine Learning"]
}

# Pandas와 상호작용
import pandas as pd
data = {
    "name": ["Alice", "Bob", "Charlie"],
    "score": [85, 90, 78]
}
df = pd.DataFrame(data)
```

(참고) 데이터 분석에서의 자료형 활용 : 집합

구분	설명	예시
중복 제거	중복 요소를 제거할 때	데이터 샘플이나 특정 결과에서 중복된 항목을 자동으로 제거
교집합, 차집합, 합집합 연산	여러 데이터셋 간 공통 / 차이를 확인	두 고객 그룹간의 공통 구매 항목 찾기
효율적인 멤버십 테스트	요소가 집합에 있는지 여부 빠르게 확인	대규모 데이터셋에서 특정 값이 존재하는지 여부를 체크
유일한 값 추출	유일값 확인	설문조사 데이터에서 중복되지 않은 응답 추출

(참고) 데이터 분석에서의 자료형 활용 : 집합

```
# 집합 생성
unique_numbers = {1, 2, 3, 4, 4, 5} # 중복된 값은 자동으로 제거됨

# 중복 제거
data_with_duplicates = [1, 2, 2, 3, 4, 4, 5]
unique_data = set(data_with_duplicates) # {1, 2, 3, 4, 5}

# 집합 연산
set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}

# 교집합 (공통 요소)
intersection = set_a & set_b # {3, 4}

# 합집합 (모든 요소)
union = set_a | set_b # {1, 2, 3, 4, 5, 6}

# 차집합 (차이 요소)
difference = set_a - set_b # {1, 2}

# 멤버십 테스트
if 3 in set_a:
    print("3은 set_a에 있습니다.")
```